

Caso 2

Integrantes:

Sergio Andrés Santana Carvajal – 202410732

Sebastian Alejandro Sarmiento Navas - 202212633

1. Actividad 1: generación de las direcciones virtuales

No se ejecutó el cifrado con datos reales: se recorrieron los índices de la matriz en el mismo orden que la función cifrar() y se calcularon las direcciones que se accederían. La matriz se almacena por filas desde la dirección 0 y el vector queda inmediatamente después. Las direcciones se calculan así:

Dirección de $m[i][j] = (i \times NC) + j$

Dirección de $v[k] = (NF \times NC) + k$

Página virtual = dirección / TP

Desplazamiento = dirección % TP

El número de páginas es $NP = (NF \times NC + TV + TP - 1) / TP$, y el número de referencias es

$NR = 3 \times NF \times NC \times 2 \times \text{numPasadas}$.

El programa escribe primero el encabezado y, en cada pasada, realiza el recorrido por filas (con el elemento $v[j \% TV]$) y el recorrido por columnas (con el elemento $v[i \% TV]$). Por cada elemento visitado se escriben tres líneas: lectura de $m[i][j]$, lectura de $v[k]$ y escritura de $m[i][j]$. Se verificó que el archivo generado es idéntico al salida.txt de ejemplo.

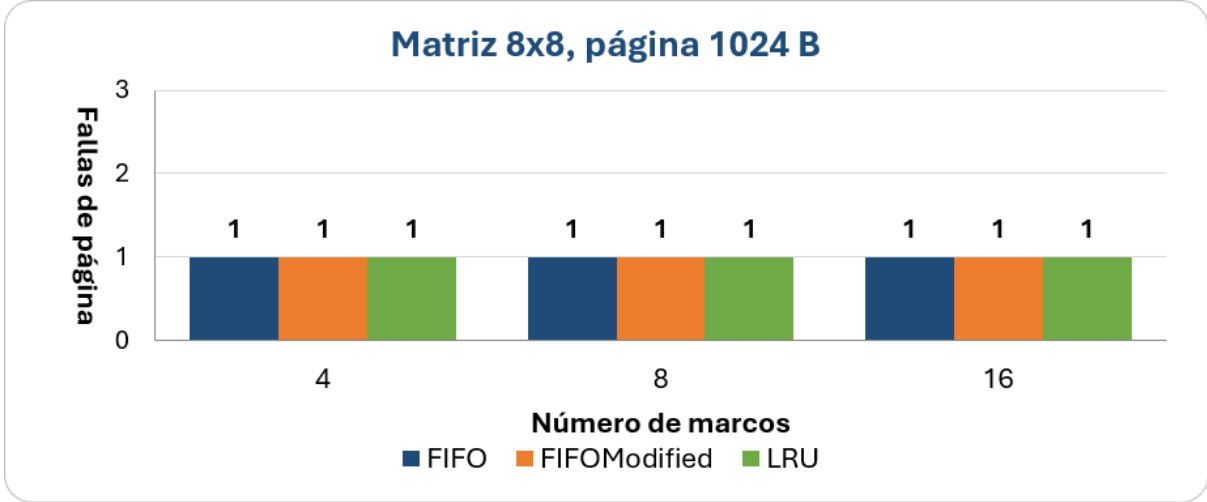
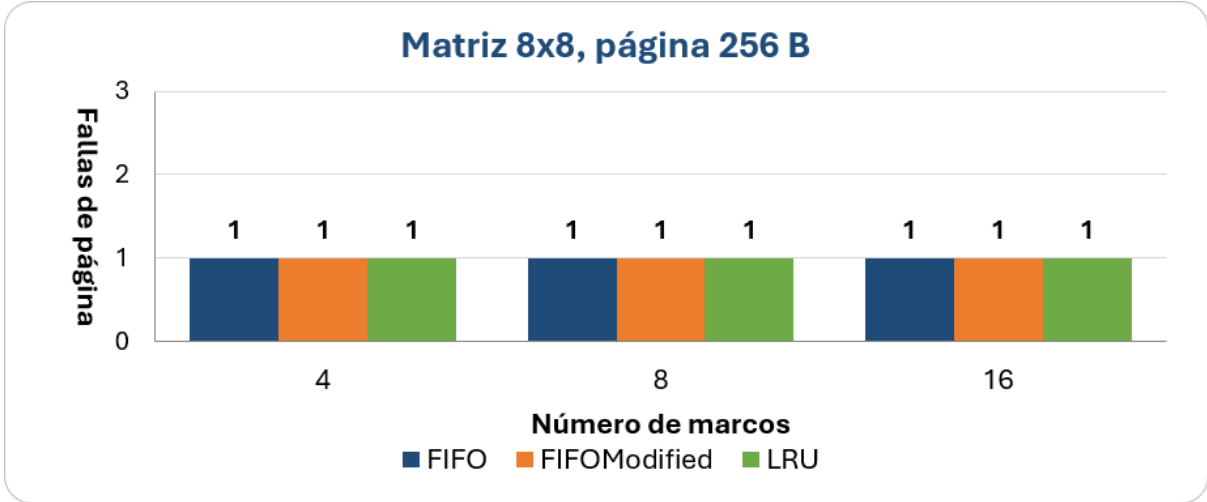
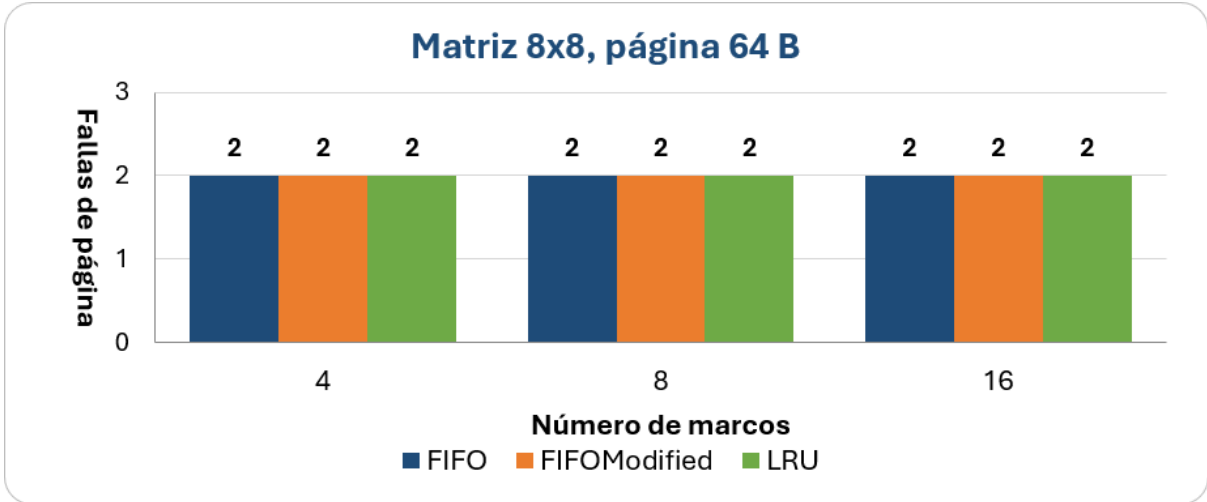
Adicionalmente, la clase EjecutorEscenarios genera los archivos de las 9 parejas matriz-página, ejecuta simulador.jar en las 81 combinaciones, lee el número de fallas y guarda en resultados.csv las fallas, los accesos exitosos y la tasa de fallas de cada escenario.

Descripción de las estructuras de datos usadas

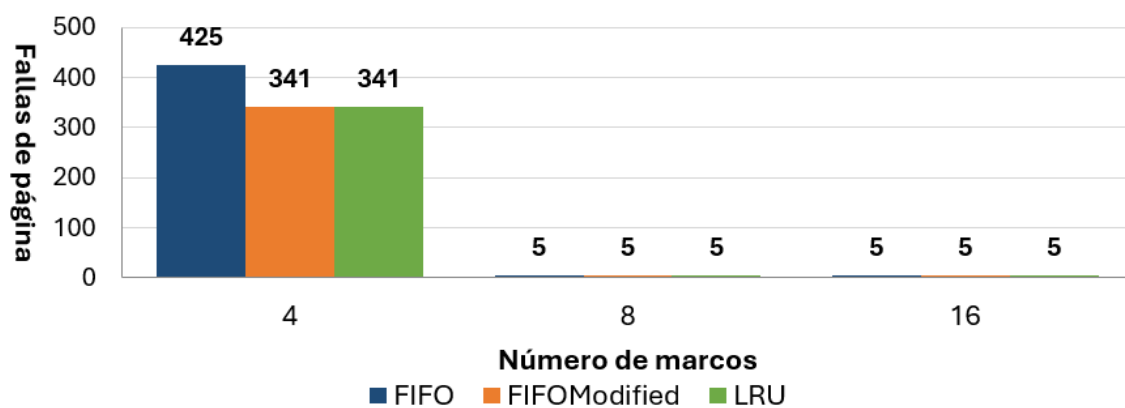
En GeneradorReferencias no se almacenan la matriz ni el vector. Solo se usan variables enteras para los parámetros y los índices, y cada dirección se calcula en el momento, por lo que el consumo de memoria es bajo. El archivo se escribe con un BufferedWriter.

En EjecutorEscenarios se usaron arreglos (`int[]` y `String[]`) con los valores a probar, un método que devuelve el número de fallos (`long`) leído de la salida del simulador, un `BufferedReader` para leer su salida y un `BufferedWriter` para escribir el archivo CSV. El único elemento que no se vio en clase es `ProcessBuilder`, que permite ejecutar desde Java el comando `java -jar simulador.jar archivo marcos politica` para automatizar las 81 pruebas.

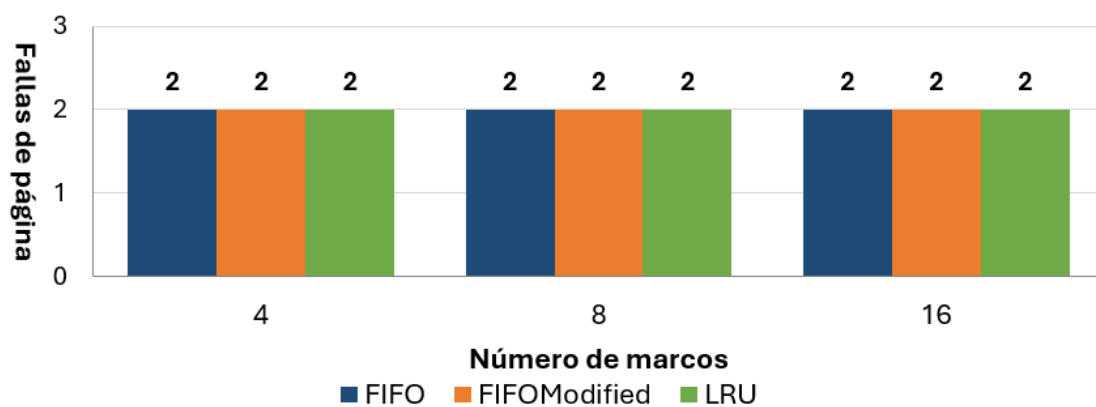
GRAFICAS



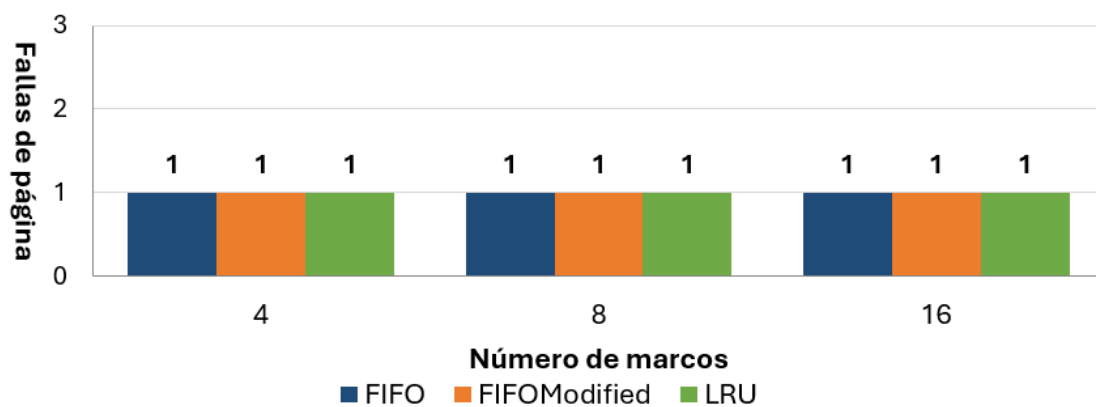
Matriz 16x16, página 64 B

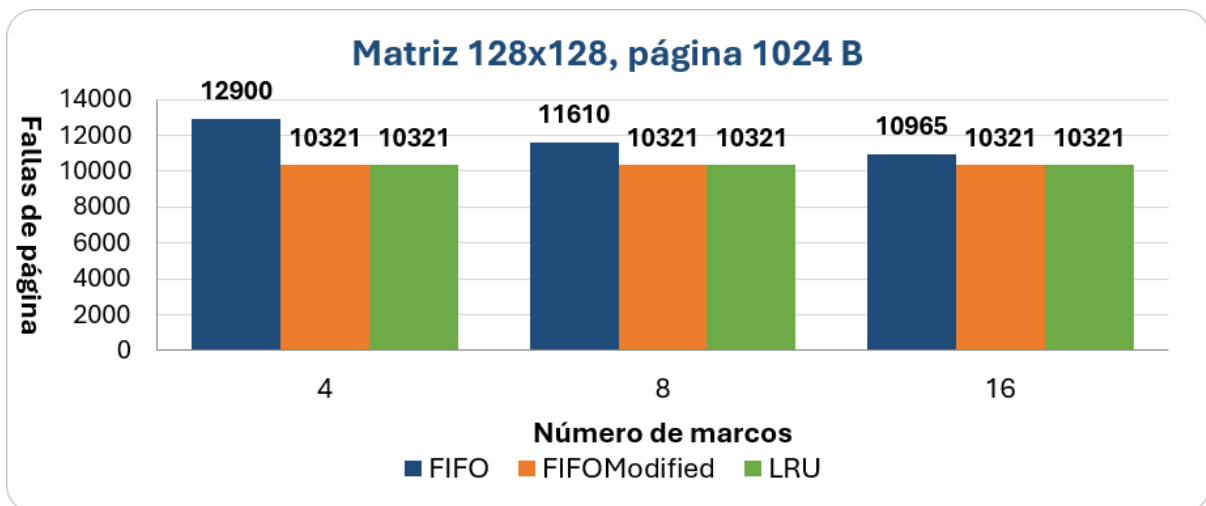
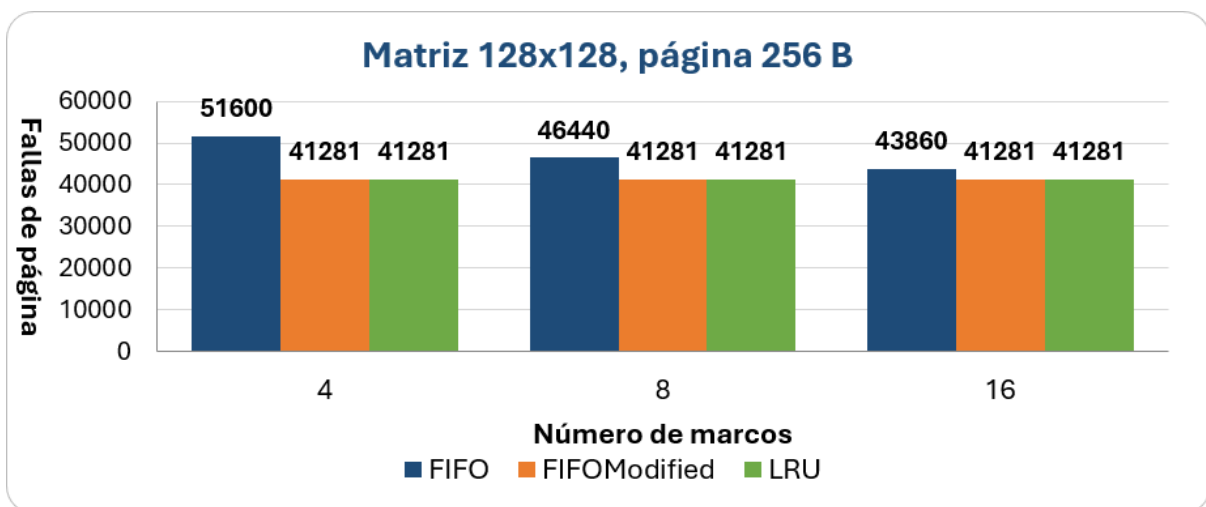
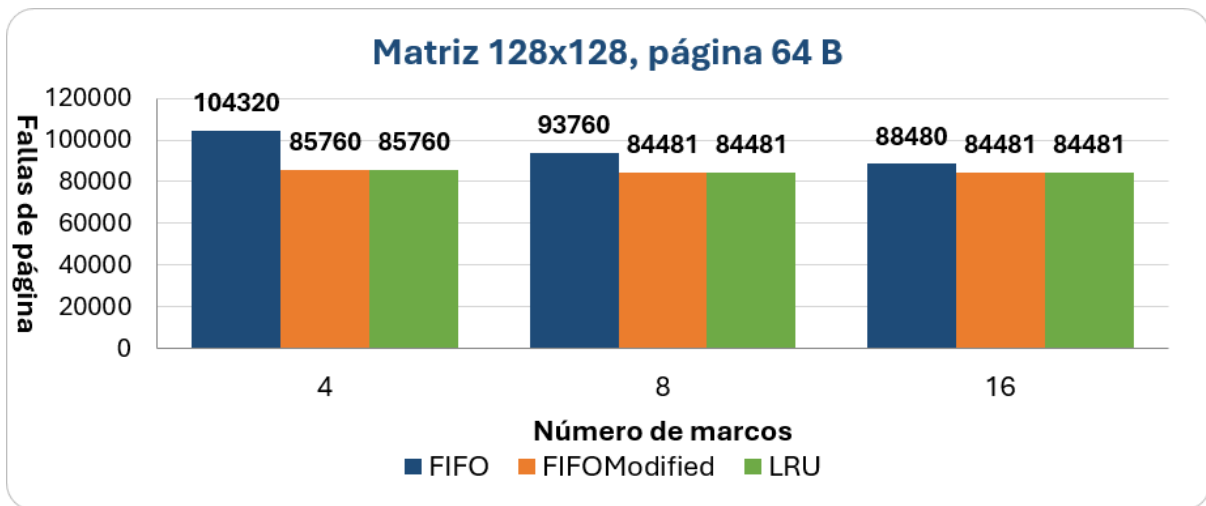


Matriz 16x16, página 256 B



Matriz 16x16, página 1024 B





Para los 81 escenarios usamos un vector de 142 bytes y 5 pasadas, los mismos valores del archivo de ejemplo, porque el enunciado no los fija. El simulador reporta un "Total accesos" que incluye el reintento después de cada falla, por eso tomamos $\text{accesos exitosos} = \text{NR} - \text{fallas}$ y $\text{tasa de fallas} = \text{fallas} / \text{NR}$.

PREGUNTAS

1. ¿Cuál política de reemplazo espera que tenga menor número de fallas en el cifrado?

Esperábamos que LRU tuviera menos fallas. En cada operación del cifrado se lee un elemento del vector clave, y como el vector es pequeño casi siempre queda en la misma página. Por ejemplo, en la matriz de 128x128 con páginas de 256 el vector está siempre en la página 64. Esa página se usa todo el tiempo, entonces LRU nunca la saca. FIFO, en cambio, saca la página que lleva más tiempo cargada aunque se esté usando, así que en algún momento saca la página del vector y le toca volver a traerla. De FIFO modificado esperábamos un resultado parecido al de LRU, porque también tiene en cuenta qué páginas se usaron hace poco.

2. ¿Lo que espera ocurre?

Sí, pero con algo que no esperábamos. LRU es la que menos fallas tiene y FIFO es la peor en todos los casos donde hay diferencia. Por ejemplo, con la matriz de 128x128, páginas de 64 y 4 marcos, FIFO tiene 104.320 fallas y LRU 85.760. Con páginas de 1024 y 4 marcos, FIFO tiene 12.900 y LRU 10.321. En las matrices de 8x8 y 16x16 casi no hay diferencia, porque casi todo cabe en memoria y solo ocurren las fallas iniciales.

Lo que no esperábamos es que FIFO modificado diera exactamente lo mismo que LRU en los 81 escenarios. Revisando la definición tiene sentido. Si cada vez que se usa una página se pasa al final de la cola, la cola queda ordenada según el último uso. Entonces la primera de la cola, que es la que se saca, es justo la que lleva más tiempo sin usarse, que es lo mismo que hace LRU.

3. Analice si páginas más grandes siempre mejoran el desempeño. Si lo hace, ¿cuál es la desventaja?

En nuestros resultados las páginas más grandes nunca empeoraron el desempeño, pero tampoco siempre lo mejoraron. En la matriz de 128x128 con LRU y 4 marcos las fallas bajan de 85.760 con páginas de 64 a 41.281 con páginas de 256, y a 10.321 con páginas de 1024. Esto pasa porque en cada página caben más filas de la matriz. Pero en la matriz de 8x8 da lo mismo usar páginas de 256 o de 1024: en los dos casos hay 1 sola falla, porque desde 256 bytes todo cabe en una página.

Las desventajas son:

- **Fragmentación interna.** En la matriz de 8x8 con páginas de 1024 los datos ocupan 206 bytes (64 de la matriz y 142 del vector), pero se reserva una página de 1024. Se desperdicia cerca del 80% de la página.
- **Más memoria física con el mismo número de marcos.** 16 marcos de 1024 bytes son 16 KB, mientras que 16 marcos de 64 bytes son solo 1 KB. Parte de la mejora viene de que el proceso está usando más RAM.
- **Más costo por falla en un sistema real.** Traer una página más grande del disco se demora más, y se cargan datos que a lo mejor no se usan.

4. Compare en el archivo de referencias los accesos producidos durante el recorrido por filas con los producidos durante el recorrido por columnas.

Escenario usado: matriz de 128x128 con páginas de 256 bytes (archivo ref_128x128_tp256.txt). Cada fila de la matriz mide 128 bytes, así que en una página caben exactamente 2 filas.

En el recorrido por filas comparamos las líneas 8 a 775 del archivo, que son las filas 0 y 1 de la primera pasada. Todos los accesos a la matriz caen en la página 0 y el desplazamiento va subiendo de 1 en 1: [mat1-0-0],0,0 → [mat1-0-1],0,1 → ... → [mat1-1-127],0,255. La página 1 solo aparece en la línea 776, con [mat1-2-0],1,0. O sea, la página cambia una vez cada 256 elementos.

En el recorrido por columnas comparamos las líneas 49160 a 49543, que son la columna 0 de la primera pasada: $[mat1-0-0], 0, 0 \rightarrow [mat1-1-0], 0, 128 \rightarrow [mat1-2-0], 1, 0 \rightarrow [mat1-3-0], 1, 128 \rightarrow [mat1-4-0], 2, 0 \rightarrow \dots \rightarrow [mat1-127-0], 63, 128$. Aquí la página cambia cada 2 elementos, y solo con la columna 0 ya se recorren las 64 páginas de la matriz (de la 0 a la 63). Después la columna 1 vuelve a empezar en la página 0. En los dos recorridos el vector está siempre en la página 64.

La diferencia se debe a que la matriz está guardada por filas (row-major). Dos elementos seguidos de una fila están en direcciones consecutivas, entonces caen en la misma página y hay buena localidad espacial. En cambio, dos elementos seguidos de una columna están separados por $NC = 128$ bytes, entonces se salta de página muy seguido. Por eso el recorrido por columnas es el que genera casi todas las fallas.

5. Usando la política LRU, compare dos ejecuciones con el mismo tamaño de matriz y el mismo número de marcos, pero con diferente tamaño de página.

Comparamos la matriz de 128×128 con 8 marcos y LRU: hay 84.481 fallas con páginas de 64, 41.281 con páginas de 256 y 10.321 con páginas de 1024. Al pasar de 64 a 256 las fallas bajan más o menos a la mitad, y al pasar de 256 a 1024 bajan a la cuarta parte.

Esto se explica por el recorrido por columnas, que es el que genera casi todas las fallas. En una columna se baja de fila en fila, y cada fila mide 128 bytes:

- Con páginas de 64, una fila ocupa 2 páginas, así que cada elemento de la columna está en una página distinta: 128 fallas por columna.
- Con páginas de 256 caben 2 filas por página, así que la página cambia cada 2 elementos: 64 fallas por columna.
- Con páginas de 1024 caben 8 filas por página: 16 fallas por columna.

Como son 128 columnas y 5 pasadas, eso da 40.960 fallas con páginas de 256 y 10.240 con páginas de 1024, solo en el recorrido por columnas. Si se suman las del recorrido por filas (64 y 16 por pasada) y la primera carga del vector, queda $40.960 + 5 \cdot 64 + 1 = 41.281$ y $10.240 + 5 \cdot 16 + 1 = 10.321$. Esos son exactamente los valores que dio el simulador. Con 8 marcos no alcanza a caber una columna completa en memoria, entonces cada cambio de página en el recorrido por columnas termina siendo una falla.

6. Si se modifica únicamente numPasadas, ¿cambian NP y NR?

NP no cambia. El número de páginas solo depende del tamaño de la matriz, del vector y de la página: $NP = \lceil (NF \cdot NC + TV) / TP \rceil$. Hacer más pasadas no ocupa más memoria; simplemente se recorren los mismos datos más veces.

NR sí cambia, y de forma proporcional. Cada pasada recorre la matriz dos veces (por filas y por columnas), y cada elemento genera 3 referencias (leer m, leer v y escribir m), entonces $NR = 6 \cdot NF \cdot NC \cdot \text{numPasadas}$. Por ejemplo, el archivo de ejemplo del enunciado (300×20 con 5 pasadas) tiene $NR = 180.000$; con 10 pasadas tendría 360.000.

Análisis del número de marcos

¿Un mayor número de marcos siempre mejora el desempeño?

No siempre, un marco adicional solo reduce las fallas si permite que las páginas que el algoritmo reutiliza quepan en memoria.

Matriz de 128x128, páginas de 64 bytes: LRU produce 85.760 fallas con 4 marcos y 84.481 con 8 y con 16. Mejora muy poco, porque el recorrido por columnas pasa por las 256 páginas de la matriz antes de repetir y ni con 16 marcos caben.

Matriz de 128x128, páginas de 1024 bytes: se necesitan 17 páginas y con 4, 8 y 16 marcos se producen 10.321 fallas.

Con FIFO sí hay mejora gradual en 128x128 con páginas de 64 bytes: 104.320, 93.760 y 88.480 fallas con 4, 8 y 16 marcos, sin alcanzar el resultado de LRU.

La desventaja es que se reserva más memoria física para un solo proceso, que deja de estar disponible para otros procesos, y cuando las páginas necesarias ya caben en memoria, más marcos no aportan mejora.

Comparación de dos ejecuciones con LRU que solo difieren en el número de marcos

Con mejora: matriz de 16x16, páginas de 64 bytes. Con 4 marcos se producen 341 fallas y con 8 marcos solo 5 (reducción de 98,5 %). El proceso tiene 7 páginas virtuales, pero solo usa 5 (las 4 de la matriz y la primera del vector, porque solo se leen $v[0]$ a $v[15]$) y el recorrido por columnas las usa de forma cíclica, con 4 marcos no caben todas y con 8 sí, por lo que solo quedan las 5 fallas obligatorias de la primera carga.

Sin mejora: matriz de 128x128, páginas de 1024 bytes. Con 4, 8 y 16 marcos se producen 10.321 fallas en los tres casos. El proceso tiene 17 páginas (16 de la matriz y 1 del vector). En el recorrido por columnas cada columna pasa por las 16 páginas de la matriz en orden y además usa la del vector todo el tiempo. Con 16 marcos, LRU deja la del vector y 15 de la matriz, y la página que saca siempre es la que se va a necesitar después, entonces cada cambio de página es una falla.

La diferencia está en si el conjunto de páginas que se reutiliza cabe o no en la memoria asignada.

Análisis cuantitativo del tiempo

Tiempo total de cada escenario

Tiempo total = $NR \times 100 \text{ ns} + \text{fallas} \times 8 \text{ ms}$. Los 81 valores están en la columna Tiempo total (s) del Excel. Ejemplos:

Matriz 8x8, página de 64 bytes: 0,016192 s

Matriz 16x16, página de 64 bytes, LRU, 4 marcos: 2,728768 s

Matriz 16x16, página de 64 bytes, LRU, 8 marcos: 0,040768 s

Matriz 128x128, página de 64 bytes, LRU, 4 marcos: 686,129152 s; con 8 o 16 marcos: 675,897152 s

Matriz 128x128, página de 64 bytes, FIFO, 4 marcos: 834,609152 s

Componente que domina el tiempo

Las fallas de página, entre 91 % y más de 99,9 % del tiempo total. Por ejemplo, en 128x128 con páginas de 64 bytes, LRU y 8 marcos, de 675,897152 s totales, 675,848 s son de fallas y solo 0,049152 s de accesos a RAM.

¿Pequeñas diferencias en fallas producen grandes diferencias en tiempo?

Sí, porque una falla (8 ms) equivale a 80000 accesos a RAM (100 ns cada uno). En 8x8, pasar de 2 a 1 falla reduce el tiempo de 0,016192 s a 0,008192 s. En 128x128 con páginas de 64 bytes y 16 marcos, la diferencia entre FIFO (88.480) y LRU (84.481) es de 3.999 fallas, unos 32 s adicionales.

DECLARACIÓN DE IA

Para este trabajo usamos inteligencia artificial como una simple herramienta de apoyo para redactar y organizar mejor las ideas, pero todo el contenido técnico es 100 % nuestro.

Antes de usar la IA, nosotros ya teníamos listo un borrador con el desarrollo completo del caso, los cálculos, el análisis de las políticas de memoria (como FIFO, LRU y las páginas) y las conclusiones. Lo único que le pedimos a la herramienta fue que nos ayudara a corregir la ortografía, unir párrafos sueltos y ordenar el texto para que se entendiera con mayor claridad, sin cambiar lo que queríamos decir ni inventar información.

Por ejemplo, le dábamos instrucciones muy específicas como:

- "Ayúdame a organizar este borrador para el informe del caso; nosotros ya hicimos todo el desarrollo, solo necesitamos que mejores la redacción y la ortografía sin cambiar las ideas ni agregar información".
- "Ayúdame a conectar estos párrafos para que no se sientan como ideas sueltas. Conserva los resultados y las conclusiones que nosotros escribimos".

También le pedimos que nos diera la tabla con los resultados del generador para poder hacer las graficas de forma más sencilla con el prompt de

“Coge todos los resultados y agruparlos en una tabla según la matriz”

¿Qué implicaciones tuvo esto?

- Cero aportes técnicos: La IA no escribió código, no hizo cálculos, no usó el simulador, no buscó referencias ni resolvió las preguntas del caso.
- Responsabilidad total: Sabemos que una buena redacción no garantiza que el contenido técnico sea correcto. Por eso, revisamos cada palabra para asegurarnos de que la herramienta no alterara el sentido de nuestros argumentos ni generalizara de más.
- Aprendizaje propio: Conceptos clave como las fallas de página, las traducciones de direcciones o las políticas de reemplazo los estudiamos, analizamos y resolvimos por nuestra cuenta.

En resumen, la IA solo funcionó como una especie de corrector de estilo y organizador gráfico de nuestras propias ideas. La autoría, el esfuerzo y la responsabilidad del informe son completamente nuestros.